



BURSA TEKNİK ÜNİVERSİTESİ
MÜHENDİSLİK FAKÜLTESİ
Bilgisayar Mühendisliği Bölümü

PROGRAMLAMA DİLLERİ

2026 Dönem Proje Ödevi

HİLAL KEPİR 23360859088



The Rust Programming Language

1. Giriş ve Dilin Amacı

Rust; Mozilla tarafından geliştirilen, C/C++ kadar yüksek performanslı ve hızlı, ancak Java gibi diller kadar bellek güvenli (memory safety) bir sistem programlama dilidir.

Rust'ı eşsiz kılan şey, arkasında sistemi yavaşlatan bir Çöp Toplayıcı (Garbage Collector) olmaması ve bellek yönetimini tamamen kod derlenirken (compile-time) halletmesidir.

2. Sözdizimi (Syntax) ve Temel Özellikler

Rust sözdizimi açısından C-ailesine (C, C++, Java) oldukça yakındır; süslü parantezler `{}` ve noktalı virgül `;` kullanımı süreklidir. Ancak, modern dillerden gelen birçok yeniliği de bünyesinde barındırır.

2.1. Değişken Tanımlama ve Değişmezlik (Immutability)

Rust'ta değişkenler varsayılan olarak **değişmezdir (immutable)**. Bu, kod güvenliğini artıran ve hataları derleme aşamasında yakalayan bir yaklaşımdır.

```
fn main() {  
    // • Değişmez Tanımlama  
    let x = 5;  
    // x = 6; // ✗ HATA: x değiştirilemez  
  
    println!("x değeri: {}", x);  
  
    // • Değişebilir Tanımlama  
    let mut y = 10;  
    println!("y başlangıç: {}", y);  
  
    y = 20; // ✅ mut sayesinde değiştirilebilir  
    println!("y güncel: {}", y);  
}
```

2.2. Veri Tipleri ve Tip Belirleme

Rust "Strongly Typed" ve "Statically Typed" bir dildir. Derleyici, değişkenin tipini çoğu zaman kullanımından çıkarabilir (Type

Inference), ancak istenirse tip açıkça belirtilebilir:

```
let tam_sayı: i32 = 42; // 32-bit işaretli tamsayı  
let ondalıklı: f64 = 3.14; // 64-bit ondalıklı sayı  
let mantıksal: bool = true;  
let karakter: char = 'R';
```

2.3. Fonksiyon Yapısı

```
fn topla(a: i32, b: i32) -> i32 {  
    a + b // Return anahtar kelimesine gerek kalmadan değer döner  
}
```

```
fn main() {  
    println!("Merhaba, Dünya!");  
}
```

Fonksiyonlar `fn` anahtar kelimesi ile tanımlanır. Rust'ta "ifade tabanlı" (expression-based) bir yapı vardır. Fonksiyonun son satırında noktalı virgül kullanılmazsa, o satır fonksiyonun dönüş değeri (return) kabul edilir.

3. Binding ve Bellek Yönetimi (Ownership & Borrowing)

Rust, bellek yönetiminde ne C'deki gibi manuel bir yöntem (malloc/free) ne de Java'daki gibi bir çöp toplayıcı (Garbage Collector) kullanır. Bunun yerine, "Ownership" (Sahiplik) adı verilen ve tamamen derleme aşamasında (compile-time) çalışan benzersiz bir sistem kullanır.

3.1. Sahiplik (Ownership) Kuralları

Rust'ta her değer bir "sahibi" (owner) olan değişkeni vardır. Bir değer aynı anda birden fazla sahibi olamaz. Değişken (sahibi) kapsam dışına (out of scope) çıktığında, bellek otomatik olarak temizlenir. Bu durum, C'deki "dangling pointer" veya Java'daki bellek sızıntısı risklerini daha kod çalışmadan ortadan kaldırır.

3.2. Ödünç Alma (Borrowing) ve Referanslar

Bir veriyi kopyalamak yerine ona erişim sağlamak için "referanslar" kullanılır. Rust bu noktada çok katı iki kural uygular:

- **Immutable Borrowing:** İstedığınız kadar salt-okunur (immutable) referans oluşturabilirsiniz.
- **Mutable Borrowing:** Aynı anda sadece bir tane değiştirilebilir (mutable) referans oluşturulabilir.

Bu kurallar sayesinde Rust, veri yarışmalarını (data races) engeller ve aynı verinin iki farklı yerden aynı anda değiştirilmesinin önüne geçer.

3.3. Yaşam Süreleri (Lifetimes)

Eğer bir referans, sahibi ölmüş bir veriye ulaşmaya çalışıyorsa (dangling reference), kod derlenmez.

```
fn main() {
    // S heap belleğinde bir String nesnesidir ve sahibi 's1' değişkenidir.
    let s1 = String::from("Merhaba");

    // Veri kopyalanmaz, sahiplik s1'den s2'ye geçer (Move).
    let s2 = s1;

    // HATA: s1 artık geçerli bir belleğe sahip değildir!
    // println!("{}", s1); <-- Bu satır derleme hatası (Compile Error) verir.

    println!("{}", s2); // Bu satır sorunsuz çalışır.
}
```

Doğru Durum (Borrowing - Ödünç Alma):

Veriyi taşımak yerine, C'deki pointer mantığına benzer şekilde ama tamamen güvenli bir referans (&) kullanarak ödünç alabiliriz:

Yukarıdaki örnek de Rust derleyicisi (scalac/cargo) belleğin durumunu daha derleme aşamasında analiz eder. s1 değişkeninin geçersiz bir bellek adresini göstermesine (Dangling Pointer) izin vermeyerek kodu reddeder. Bu sayede çalışma zamanında (runtime) oluşabilecek çökmeler (crash) sıfıra indirgenmiş olur.

4. ADT (Abstract Data Types) Yapısı, Tipler ve Generic

Rust, güçlü bir tip sistemine (Type System) sahiptir. OOP'deki sınıf (class) hiyerarşileri yerine, **Algebraic Data Types (ADT)** prensiplerine dayanan ve veriyi davranıştan ayıran bir mimari sunar.

4.1. ADT ve Gelişmiş Enum Yapısı

Rust'ta soyut veri tipleri oluşturmak için struct ve enum yapıları kullanılır. Geleneksel dillerin aksine Rust'ta enum, sadece basit birer sabit listesi değil; içlerinde

farklı veri tiplerini barındırabilen güçlü birer **Sum Type** (Toplam Tipi) örneğidir.

Bu yapı, özellikle hata yönetimi (Result<T, E>) ve boş değer kontrolü (Option<T>) gibi kritik mekanizmaların temelini oluşturur. Rust'ta null kavramı yoktur; bunun yerine bir değer varlığı veya yokluğu Option ADT yapısı ile derleme aşamasında zorunlu olarak kontrol ettirilir.

```
// İçerisinde farklı veri tipleri taşıyan gelişmiş bir ADT (Enum) tanımı
enum RobotKomutu {
    Dur, // Verisiz eleman
    Yuru { x: i32, y: i32 }, // İsimlendirilmiş alanlar (Struct gibi)
    MetniYaz(String), // Tekil veri taşıyan eleman
}

fn komutu_calistir(komut: RobotKomutu) {
    // match ifadesi ile ADT'nin tüm ihtimalleri derleme aşamasında kontrol edilir
    match komut {
        RobotKomutu::Dur => println!("Robot durduruldu."),
        RobotKomutu::Yuru { x, y } => println!("Robot {}:{} konumuna yürüyor.", x, y),
        RobotKomutu::MetniYaz(mesaj) => println!("Robot ekranına yazılıyor: {}", mesaj),
    }
}
```

4.2. Genericlik ve Trait Sistemi

Rust, kod tekrarını önlemek için **Generic** (<T>) tanımlamaları destekler. Ancak Java'daki (Kalıtım/Inheritance) yerine, **Trait** soyutlama mekanizmasını kullanır.

Trait'ler, bir tipin hangi metotlara sahip olması gerektiğini tanımlar (Java'daki Interface yapısına benzer). Generic fonksiyonlar yazılırken, "Trait Bounds" kullanılarak parametre olarak gelecek generic tipin hangi özelliklere sahip olması gerektiği derleyiciye bildirilir.

5. Nesneye Yönelik (OOP) Kavramların Karşılanması

Rust, geleneksel nesne yönelimli programlama dillerinden (Java, C++ gibi) farklı bir tasarım felsefesine sahiptir. Sınıf (class) tabanlı bir hiyerarşi yerine, **"Kalıtım yerine Kompozisyon" (Composition over Inheritance)** prensibini benimser. Nesneye yönelik temel kavramları şu mekanizmalarla çözer:

5.1. Kapsülleme (Encapsulation)

Rust'ta veri ve davranış (metotlar) birbirinden ayrı tanımlanır. Veriler struct yapıları içinde tutulurken, bu verilere ait metotlar impl (implementation) blokları içine

yazılır. Kapsülleme ve erişim kontrolü, pub (public) anahtar kelimesiyle sağlanır. pub ile işaretlenmeyen alanlar ve metotlar varsayılan olarak gizlidir (private) ve sadece tanımlandıkları modül içinden erişilebilir.

5.2. Çok Biçimlilik (Polymorphism) ve Soyutlama

Çok biçimlilik, Rust'ta **Trait** yapısı ve **Generic** tipler kombinasyonu ile çözülür. Java'daki interface yapısına benzeyen Trait'ler, farklı nesnelerin ortak bir arayüz üzerinden farklı davranışlar sergilemesini sağlar.

Rust bu noktada iki tür çok biçimlilik sunar:

1. **Statik Çok Biçimlilik (Static Dispatch):** Derleme aşamasında generic tiplerin çözülmesidir. Performans kaybı sıfırdır.
2. **Dinamik Çok Biçimlilik (Dynamic Dispatch):** Çalışma zamanında (runtime) hangi metodun çağrılacağına karar verilmesidir. dyn anahtar kelimesi ve akıllı işaretçiler (Box, Rc) kullanılarak polimorfik nesne dizileri oluşturulabilir.

```
1 // Kapsülleme (Encapsulation): Veri yapısı tanımlanır
2 pub struct BankaHesabi {
3     sahip: String,
4     pub bakiye: f64, // Dışarıdan doğrudan erişilebilir
5     gizli_id: u32, // Sadece bu module özel (Private)
6 }
7
8 // Davranışların (Metotların) Veriye Bağlanması
9 impl BankaHesabi {
10     // Yapıcı (Constructor) Metot
11     pub fn yeni(sahip: String, ilk_bakiye: f64) -> Self {
12         BankaHesabi { sahip, bakiye: ilk_bakiye, gizli_id: 12345 }
13     }
14
15     // Nesne metodudur (&self parametresi nesnenin kendisine işaret eder)
16     pub fn para_yatir(&mut self, miktar: f64) {
17         self.bakiye += miktar;
18     }
19 }
20
21 // Çok Biçimlilik (Polymorphism) İçin Trait Tanımı
22 pub trait OdemeSistemi {
23     fn ode(&self, tutar: f64);
24 }
25
26 struct KrediKarti;
27 struct Kripto;
28
29 // Farklı struct'lar aynı Trait'i farklı şekilde uygular (Polymorphism)
30 impl OdemeSistemi for KrediKarti {
31     fn ode(&self, tutar: f64) { println!("Kredi kartı ile {} TL ödendi.", tutar); }
32 }
33
34 impl OdemeSistemi for Kripto {
35     fn ode(&self, tutar: f64) { println!("Kripto cüzdanından {} TL gönderildi.", tutar); }
36 }
37
38 fn main() {
39     let mut hesapim = BankaHesabi::yeni(String::from("Hilal"), 1000.0);
40     hesapim.para_yatir(500.0);
41
42     // Dinamik Çok Biçimlilik Kullanımı (Polimorfik Vektör/Liste)
43     let odeme_yollari: Vec<Box<dyn OdemeSistemi>> = vec![
44         Box::new(KrediKarti),
45         Box::new(Kripto),
46     ];
47
48     for sistem in odeme_yollari {
49         sistem.ode(hesapim.bakiye);
50     }
51 }
```

6. Paralellik ve Eşzamanlılık (Concurrency & Parallelism)

Rust, "Korkusuz Eşzamanlılık" (Fearless Concurrency) felsefesine sahiptir. Veri yarışmalarını (data races) çalışma zamanında değil, derleme aşamasında (ownership ve borrowing kuralları sayesinde) engeller. Bir verinin aynı anda hem birden fazla thread tarafından paylaşılmasına hem de değiştirilmesine derleyici izin vermez.

İş parçacıkları (threads) arasındaki veri iletimi ve paylaşımı için standart kütüphanedeki Send ve Sync trait'leri ile Mutex (Mutual Exclusion) yapıları kullanılır.

7. Fonksiyonel Programlama Yapıları

Rust, saf bir fonksiyonel dil olmasa da birinci sınıf fonksiyonel özellikleri destekler. Fonksiyonlar değişkenlere atanabilir, başka fonksiyonlara parametre olarak geçilebilir veya geri döndürülebilir.

Iterator yapısı, map, filter, fold gibi fonksiyonel metotları sıfır maliyetli soyutlama ile sunar.

8. Dilin Sözdizimsel (Syntax) ve Anlamsal (Semantic) Özellikleri

- **İfade Tabanlılık:** Rust'ta neredeyse her yapı (if-else, match, bloklar) bir değer üretir. Fonksiyonlarda return kelimesi yerine son satırdaki ifadenin sonuna noktalı virgül koymamak yeterlidir.
- **Güvenli Semantik:** Tip dönüşümleri asla gizli (implicit) yapılmaz; as anahtar kelimesiyle açıkça (explicit) belirtilmesi zorunludur.